AgentVax: A Self-Immunising Wallet Agent with Adaptive Financial Autonomy

[Author 1], [Author 2], [Author 3]

Team [team name], [university, country]

2026 Shenzhen International FinTech Competition (FinTechathon), International Track, Topic F: Agent Wallet

Contact: [email]

Abstract

AI agents that hold wallets can be talked into spending money they should not spend, and several have already lost real funds. Existing agent wallets ask what the agent is allowed to do and enforce it with caps and allowlists. AgentVax asks a second question: how much of its authorised power should the agent exercise on its own right now? The owner states a mission in plain language; it becomes a signed mandate on chain that the model cannot change. Before every payment the wallet contract checks three things: is it allowed, can the owner afford it, and is it safe right now, and a daily caution allowance limits how many borderline payments may pass without the owner being asked. When one wallet is attacked it publishes an immunity capsule, a structured alert with no private data that describes the attack by its features, so other wallets recognise reworded versions and become more cautious at once. A detector trained jointly with federated learning then gives lasting protection without any wallet sharing its data. Learning and shared intelligence can only reduce autonomy. Only the owner can extend authority, and only after a delay chosen in advance, so that an owner under pressure cannot be rushed into a mistake. The paper specifies the architecture, the algorithms, the security design and a reproducible evaluation protocol whose headline measure is the share of legitimate tasks completed without the owner, with zero policy violations. It is a design and protocol document; measured results are reported in the accompanying execution evidence.

***Keywords*—**agent wallet, adaptive autonomy, prompt injection, payment fraud, threat sharing, federated learning.

1. Introduction

The theme of the International Track is "AI as a Financial Participant": an agent that completes real tasks in a financial scenario [1]. Topic F asks for a wallet agent that autonomously performs asset management, transaction execution and risk control on chain. Entries are scored on task completion (40%), security and compliance (30%) and innovation and interaction (30%), with a 0–5 point bonus for on-chain exploration [1].

A wallet agent differs from a chatbot in one respect that matters for security: its output moves value. A language model reads text from many sources and cannot reliably tell data from instructions [2, 3]. Table 1 lists four public incidents in which an AI agent with a wallet sent funds it should not have sent. In the most recent, an attacker posted an instruction in Morse code; one model translated it and a wallet bot executed the result [4, 5]. In the laboratory, Patlan et al. showed that planting false entries in a wallet agent's memory is more effective still [6]. Regulators have named the same risk for agent-initiated payments [7]. An agent that pays invoices also inherits the oldest payment fraud there is: a message that appears to come from a known supplier and asks for payment to a new account. That fraud usually works by rushing a person, not by fooling a machine.

Two observations follow. First, the failures are of different kinds: persuasion, injection, and an arithmetic mistake with no attacker at all. A wallet that only asks whether a payment is permitted misses the third kind entirely. Second, each agent failed alone. Nothing learned in one incident protected the next wallet, because the evidence is conversation and transaction data that owners cannot hand to others.

Contributions. AgentVax combines:

1. a *mission-to-mandate* step and a task engine that completes four wallet tasks from plain-language instructions;
2. a *three-question authorisation rule*: the tier of a request is the highest tier demanded by permission, affordability and current threat level, enforced by the contract and a guardian co-signature;
3. a *financial state engine* that makes affordability a condition of autonomous spending and is computed on chain from registered obligations;
4. a *caution allowance*: a daily budget of tolerated uncertainty that shrinks at once under threat and recovers slowly;
5. *immunity capsules*: feature-based, expiring, tighten-only alerts that match reworded attacks and inform the owner at the moment of approval;
6. a *federated detector* that turns each attack into lasting protection, with every shared model anchored on chain;
7. *owner-pressure safeguards*: time-locked expansion of authority, holds triggered by pressure, and considered confirmation;
8. an *evaluation protocol* and attack suite (VaxLab) whose headline measure is the safe autonomy rate.

Scope. The prototype uses three wallets on a public EVM test network with valueless test tokens. Items left for later are listed in Section 14.

2. Background and Related Work

Agent wallets. Account abstraction (ERC-4337) lets a smart-contract account define its own validation logic, including limited session keys [11]. Agent identity and reputation registries have been proposed in ERC-8004 [12]. Commercial agent wallets add caps, daily limits and allowlists. These

Table 1. Reported incidents involving AI agents that control wallets.

Date	Agent	What happened	Reported loss	Kind of failure
Nov. 2024	Freysa [8]	A player persuaded the agent that its transfer-approval function handled incoming funds; it released the prize pool. This was a designed game.	About $47,000	Persuasion of the model
Mar. 2025	AIXBT [9]	An attacker entered the control dashboard and queued two prompts instructing the agent to transfer funds.	55.5 ETH, about $106,000	Injected instructions after a dashboard breach
Feb. 2026	Lobstar Wilde [10]	The agent intended to send a small gift and transferred its entire token holdings.	About $250,000 in tokens	Agent error in the amount
May 2026	Grok and Bankr [4, 5]	A token transfer raised the wallet's permissions; a Morse-coded post was translated into a transfer order and executed.	About $150,000–$174,000; most reportedly returned	Encoded prompt injection

Table 2. A typical agent wallet compared with AgentVax.

Aspect	Typical agent wallet	AgentVax
Central question	What is the agent allowed to do?	How much of its authorised power should it exercise on its own right now?
How the owner sets it up	Individual limits and lists	A mission in plain language, converted to a mandate the owner signs and the model cannot alter
Checks before paying	Is this allowed?	Allowed, affordable, and safe right now; the strictest answer wins
Knowledge of the owner's finances	None; a cap is a fixed number	Reserve and upcoming bills are registered on chain; the spendable amount is recomputed for every request
Many slightly unusual small payments	Each passes on its own	They draw on a daily caution allowance; when it runs out the rest wait for the owner
Who decides the risk level	The model, or a filter in the same process	A separate guardian whose signature the contract requires and the model cannot produce
When another wallet is attacked	Nothing changes	A capsule raises the tier of matching requests, including reworded ones, and is shown to the owner when approval is asked
What wallets share	Nothing	Fixed-field alerts and masked model updates; never messages, prompts or transactions
Effect of a good track record	May relax scrutiny	Lowers risk estimates only; it never creates a permission
The owner under pressure	The owner's approval is final and immediate	Approvals of pressured requests are held, and the owner can cancel at once
Who may loosen a rule	Often the agent's own key	Only the owner, and only after a delay fixed in advance; tightening is instant

bound the loss per wallet, but they treat autonomy as fixed, ignore the owner's financial position and provide no shared defence.

Prompt injection. Indirect injection places instructions in content the agent retrieves [2]; it heads the OWASP list of risks for language-model applications [3]. Architectural defences separate a privileged model that plans from a quarantined model that reads untrusted content [13], or restrict what data may flow into actions [14]. AgentVax adopts this separation.

Threat sharing. Security teams exchange structured indicators of attack in formats such as STIX [15], and banks exchange fraud alerts. The format of a capsule is therefore not new. What is new is the effect: a capsule is applied automatically by a wallet contract and changes what an agent may spend without approval.

Federated learning. Clients train locally and a coordinator averages their updates [16, 17], with secure aggregation [18] and robust aggregation [21]; FATE [19] and Flower [20] implement these.

Table 2 summarises how AgentVax differs from a typical agent wallet.

3. Requirements and Design Goals

Table 3 maps each scoring criterion to the component that addresses it and to the evidence submitted. Three principles govern the design:

P1. The model proposes; deterministic code authorises; the wallet contract enforces.

P2. Learning changes risk estimates; only the owner changes authority.

P3. Caution is immediate and expansion is slow, for the model, the network and the owner alike.

4. System Architecture

Figure 1 shows the components. An instruction from the owner reaches the *planner*, the only model allowed to propose transactions. External content is read only by the *quarantined reader*, which returns typed fields and flags and never free text. The planner emits a structured *intent*. The *validator* checks it deterministically. The *guardian* scores the associated content with the injection detector and signs the intent together with a risk band, its source and its flags. The *policy wallet* contract then decides: it checks the mandate, computes the

Table 3. Alignment of AgentVax with the official scoring criteria [1].

Criterion	Weight	AgentVax component	Evidence submitted
Task completion	40%	Mission-to-mandate; task engine for four wallet tasks; financial state engine for asset management; clarification of ambiguous requests	Transaction hashes for each task; safe autonomy rate; task-suite and affordability-suite results; test cases in the repository
Security and compliance	30%	Three-question tiers and caution allowance enforced in the contract; guardian co-signature; planner and reader separation; tighten-only capsules; time-locked owner actions and pressure holds; testnet-only operation	Security self-assessment report; VaxLab attack results; known-risk list (Table 12)
Innovation and interaction	30%	Adaptive autonomy; feature-based capsules; federated detector; protection of the owner under pressure; an interface that explains each decision and requires considered confirmation	Capsule response on reworded attacks; before/after detection on an unseen attack family; demo video
Challenge bonus	0–5 pts	Mandate, obligation registry, allowance, capsule bulletin and model anchoring on chain; time-locked and co-signed execution paths	Contract addresses and transaction hashes for each

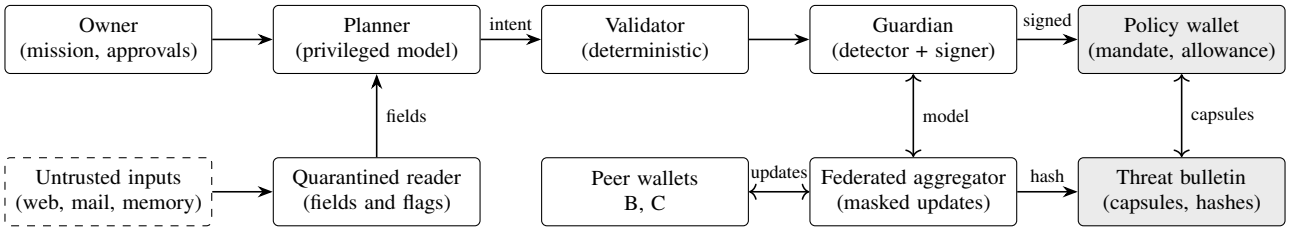


Figure 1. AgentVax architecture. Grey boxes are smart contracts; the dashed box is untrusted input. Only the planner proposes transactions, and it never reads untrusted text directly. The wallet contract decides from the mandate, its obligation registry, its allowance and the capsules in the bulletin.

discretionary amount from its *obligation registry*, reads active capsules from the *threat bulletin* contract, and charges the caution allowance. The detector is trained jointly through the *federated aggregator*.

5. Mission, Mandate and Tasks

5.1 From mission to mandate

The owner does not authorise each transaction. The owner states a mission once, for example: "Keep 600 in reserve. Pay my landlord and my approved suppliers on time. Pay approved payees up to 500 without asking. Ask me about anyone new. Do not borrow, invest or trade." The model converts this into a structured *mandate* (Table 4) and shows it back in plain language. The mandate takes effect only when the owner signs it, it is stored in the wallet contract, and no model output can modify it. It carries an expiry date, after which the agent can read but not pay, and two waiting times that protect the owner (Section 7.4).

5.2 Authority is not familiarity

An approved payee is a pair: an identity and a destination address. A request to pay a known payee at a new address is a request to approve a new payee, however many successful payments preceded it. History is used to estimate risk, for example the usual amount for a payee. It can lower or raise an estimate; it cannot add a payee, raise a limit or shorten a delay.

5.3 Supported tasks

Table 5 lists the four tasks. They cover transaction execution (T1, T2), asset management (T3, T4) and risk control (the decision applied to all four).

Table 4. Example mandate derived from a mission.

Field	Value
Reserve R	600
Autonomous limit per payment c_1	500
Autonomous limit per day d_1	1,000
Limit with delay c_2	2,000
Approved payees (identity and address)	Landlord, hosting provider, software vendor
New payee or new address	Owner approval
Prohibited actions	Borrowing, investing, trading
Caution allowance per day A	10 points
Loosening delay Λ	24 hours
Pressure hold Π	60 minutes
Trusted co-signer	Optional
Expiry	90 days

5.4 Typed intents and explanation

The planner does not produce transactions. It produces an intent

$$I = (\mathit{task}, \mathit{payee_id}, \mathit{amount}, \mathit{asset}, \mathit{schedule}, \mathit{purpose}, \mathit{nonce}),$$

serialised against a fixed schema. *payee_id* is an index into the on-chain allowlist; the planner cannot supply a raw address. If a required field cannot be filled, the agent asks one question instead of guessing. After each action the interface shows the task, the tier, the transaction hash and which check determined the outcome. After a delay or refusal it states the reason in

Table 5. Wallet tasks supported by the prototype.

ID	Task	Example instruction and on-chain effect
T1	Payment	"Pay the hosting invoice." Transfers test stablecoin to an approved payee.
T2	Recurring payment	"Pay rent on the 1st of each month." Creates a schedule, which also registers the next payment as an obligation.
T3	Idle-balance sweep	"Save what I do not need." Moves only the discretionary amount to a savings vault.
T4	Fee-reserve upkeep	"Never let fees run out." Tops up the native-token reserve when it falls below a floor.

the owner's terms, for example the size of a shortfall, and the available choices.

6. Financial State Engine

Caps answer whether a payment is permitted. They do not answer whether it is wise today. The state engine supplies that answer from three on-chain quantities: the stablecoin balance B; the reserve R; and a registry of obligations $O = \{(a_j, d_j)\}$, each an amount and a due date. Scheduled payments (T2) enter the registry automatically; one-off bills are registered when the owner or the agent records them.

6.1 Discretionary amount

For a request made at time t with planning horizon H, the discretionary amount is

$$D \;=\; B \;-\; R \;-\; \sum_{j \in O,\; d_j \le t+H} a_j,$$

where an obligation is left out of the sum when the request is the payment of that obligation. A request of amount a is classified as

$$\mathit{afford}(a) = \begin{cases} \text{comfortable} & a \le D, \\ \text{tight} & D < a \le B - R, \\ \text{breach} & a > B - R. \end{cases}$$

A comfortable request does not raise the tier. A tight request is queued, and a request that would breach the reserve needs the owner. Expected income is shown to the owner but never counted, so the agent cannot spend money that has not arrived.

6.2 Where it is enforced

The wallet contract computes the discretionary amount itself at execution time, so affordability does not depend on the model's arithmetic. The registry is bounded in length to keep the cost predictable. The agent may add an obligation, which lowers what it can spend; only the owner may remove one or lower the reserve.

6.3 Worked example

Table 6 shows a wallet holding 1,850 under the mandate of Table 4. With three bills due, $D = 180$. The agent is asked to upgrade a hosting plan for 300. The payee is approved, the amount is under the limit and there is no sign of attack, so a typical agent wallet would pay. Here $D < 300 \le B - R = 1{,}250$, so the request is tight. The agent replies that the payment is allowed but would leave the wallet 120 short of its bills unless the expected 500 arrives, queues it, and offers two choices: wait for the income, or approve now.

Table 6. Worked example of the financial state.

Item	Amount
Balance B	1,850
Reserve R	−600
Rent, due in 5 days	−800
Hosting invoice	−150
Software bill	−120
Discretionary amount D	180
Expected income (shown, not counted)	500

The same check would have stopped the third incident in Table 1, where no attacker was involved: an amount far above the discretionary amount cannot execute without the owner.

7. Security Design

7.1 Threat model

The adversary controls content the agent may read: web pages, messages, invoices, token metadata, and entries that reach the agent's memory. The adversary may operate one participating wallet and publish capsules or model updates from it, and may use AI of its own to vary and adapt attacks. The adversary may also contact the owner directly and press for an approval or a change of rule. The adversary does not hold the owner key or the guardian key and cannot alter deployed contracts. No defence assumes that the agent's model out-reasons the attacker: a fully manipulated model must still be unable to move funds beyond the mandate.

7.2 Three-question tiers

Table 7 gives the rule. Each question proposes a tier and the request receives the highest tier any of the three questions demands (Algorithm 1). The agent holds a session key valid only for calls that satisfy the rule; the owner key can revoke it or freeze the wallet at any time.

The detector runs off chain, so the contract cannot observe its output. The guardian therefore signs the intent hash together with the risk band, the source of the request and its flags, and the contract requires a valid guardian signature with band low before it will execute at L1. The planner has no access to the guardian key, so a manipulated planner cannot promote its own request.

7.3 Caution allowance

The three questions judge one request at a time. A patient attacker can send many requests that each look almost normal. The caution allowance bounds how much uncertainty the wallet accepts in a day without the owner.

Each L1 request is charged points for the flags it carries (Table 8). The flags are checkable facts, not model opinions: the contract computes those it can see and takes the rest from the guardian's signed attestation. If the remaining allowance covers the charge, the request executes and the allowance is reduced; otherwise the request is queued at L2. The point values are owner-set integers; the table shows the defaults we chose, which are not learned.

Table 7. Permission tiers. A request receives the highest tier demanded by any of the three questions.

Tier	Allowed?	Affordable?	Safe right now?	Authorisation required
L0 Read	No value moves	Not applicable	Not applicable	None
L1 Routine	Approved payee; amount $\le c_1$; daily total $\le d_1$	Comfortable: $a \le D$	Detector band low; no pressure flag; no capsule fully matches	Session key, guardian signature, and enough allowance
L2 Elevated	Approved payee; amount $\le c_2$	Tight: $D < a \le B - R$	Detector band medium, a pressure flag, or a single-observer capsule matches	Queued for delay Δ; owner may cancel
L3 Sensitive	New payee or address; amount $> c_2$; prohibited action; any loosening of the mandate	Breach: $a > B - R$	Detector band high, or a confirmed capsule matches	Owner co-signature; held for Π under pressure; rule changes wait Λ

Table 8. Default charges against the caution allowance.

Flag on an otherwise routine request	Points
None: a payee paid before, usual amount, owner instruction	0
Request arose from external content, not a direct owner instruction	2
Request uses more than half of the discretionary amount	2
Amount exceeds the largest amount previously paid to this payee	3
First payment to this payee since approval	3
Request partially matches an active capsule	5

Algorithm 1 Authorisation of intent I, amount a

if schema, provenance or state check fails **then return** reject
end if
$q_1 \leftarrow$ tier from the mandate ▷ allowed?
$q_2 \leftarrow$ tier from *afford*(a) ▷ affordable?
$q_3 \leftarrow$ tier from guardian band; at least L2 if a pressure flag is set
for each unexpired capsule c that fully matches I **do**
    $q_3 \leftarrow \max(q_3,\ \mathit{restriction}(c))$ ▷ safe now?
end for
tier $\leftarrow \max(q_1, q_2, q_3)$
if *tier* $=$ L1 **then**
    $\kappa \leftarrow$ points charged for the flags of I
    if $\kappa \le \alpha$ **then** $\alpha \leftarrow \alpha - \kappa$
    else *tier* $\leftarrow$ L2 ▷ allowance spent
    end if
end if
execute, queue or request co-signature by *tier*
hold an owner approval for Π if a pressure flag or capsule is present
report *tier*, the deciding check and any matching capsule

The allowance itself moves with circumstances. It is halved the moment a capsule that applies to the wallet is published, and set to zero when the wallet's own owner confirms an attack. It then refills at a fixed rate ρ up to the mandate value A:

$$\alpha(t) = \min\big(A,\ \alpha(t_0) + \rho\,(t - t_0)\big).$$

With the defaults, an attacker who sends a series of invoices from an approved payee, each slightly above the previous maximum, is charged five points per invoice; the third one waits for the owner.

7.4 The owner under pressure

The top tier relies on the owner, and the owner can be rushed, frightened or deceived. Most payment fraud succeeds this way: the victim authorises it. Banks already answer with time. Banks in Singapore delay the activation of a new security token by at least twelve hours and apply cooling-off periods to key account changes [24], and United Kingdom rules allow a payment to be held for up to four business days when fraud is suspected [25]. AgentVax builds this protection into the wallet as a commitment the owner makes while calm and cannot waive in the moment [26].

Loosening is slow. Any owner action that expands authority takes effect only after the loosening delay Λ fixed in the mandate: adding a payee or an address, raising a limit or the allowance, lowering the reserve, or shortening a delay, including Λ itself. Cancelling a pending change, lowering a limit and freezing the wallet are immediate. The contract enforces this as a time-lock on owner calls. The same rule limits the damage of a stolen owner key: the thief must wait, and the real owner can freeze the wallet.

Pressure is a signal. Urgent wording, a request for secrecy and an instruction to ignore earlier details are *pressure flags*, raised by a fixed phrase list and by the reader. A request that carries a pressure flag cannot execute at L1, and an owner approval given while a pressure flag or a matching capsule is present takes effect only after the pressure hold Π. The agent states the reason: the message presses for immediate payment, which is a common sign of fraud. The hold has a fixed length. An attacker who removes the pressure to avoid the hold gives up the main lever of the fraud.

Considered confirmation. For an approval that carries flags, the interface shows the payee's current and requested address side by side, the payment history and any matching capsule, and asks the owner to type the payee's name and to state who asked for the payment. The answers can lengthen a hold; nothing the owner types in the moment can shorten one. To limit fatigue, these steps appear only for flagged L3 requests.

Trusted co-signer. Genuine emergencies exist. The mandate may name a second person whose signature, together with the owner's, releases a hold or a pending change early. The co-signer cannot act alone.

7.5 Hallucination protection

The validator applies three deterministic checks before the guardian sees an intent: (i) *schema*: all fields present and well typed; (ii) *provenance*: *payee_id* exists in the on-chain allowlist, and *amount* and *asset* equal a value in the owner's instruction, in the mandate, or in a typed field returned by the reader for a document the owner referenced; (iii) *state*: the nonce is fresh and the balance covers the amount and the fee. Balances, obligations and transaction status shown in the interface are read from the chain, not generated by the model.

7.6 Injection defence

Defence is layered so that no single component is trusted:

Table 9. Fields of an immunity capsule.

Field	Values
Attack family	Direct, indirect, memory, encoded, recipient substitution
Channel	Owner chat, web page, message, memory, token metadata
Features	Any of: known payee identity, new destination, urgency, secrecy request, override attempt, encoded content, amount anomaly, memory-sourced
Affected action	Payment, schedule creation, sweep, payee or mandate change
Restriction	Delay (L2) or owner approval (L3)
Confidence	Single observer, or confirmed by q wallets
Expiry	Time after which the capsule has no effect
Guard version	Detector version in force when observed
Attestations	Signatures of the observing wallets

1. *Separation.* The reader has no tools and returns only enumerations, numbers, allowlist indices and flags. Free text from untrusted sources never enters the planner's context [13, 14].
2. *Bounded memory.* The planner sees only what the current decision needs. Long-term memory holds structured facts (per-payee address, usual amount, last verification) and threat records; raw prompts and conversations are not retained.
3. *Detection.* The guardian scores the raw untrusted content associated with a request and assigns a risk band (Section 9).
4. *Containment.* Whatever the model outputs, the contract bounds the result. An undetected attack can at most cause L1 payments to approved payees, within the limits, the discretionary amount and the day's allowance.
5. *Shared alert.* Capsules narrow that remaining window while a threat is active (Section 8).

8. Immunity Capsules

Retraining a detector takes time and needs examples. A wallet that has just been attacked can warn the others at once. An immunity capsule is a small record of fixed fields that says what kind of attack was seen, which features it had, which action it targeted, and how much autonomy to withdraw and for how long.

8.1 Contents

Table 9 lists the fields. Every field is an enumeration, a set of flags, a number or a signature. A capsule carries no message text, no prompt and no transaction, so it cannot leak private data and cannot itself carry an injected instruction.

8.2 Matching reworded attacks

A capsule describes an attack by its features, not its wording. Let F_c be the capsule's feature set and F_I the flags of a request. The capsule *fully matches* when its channel is in the wallet's on-chain profile, its affected action equals the request's action and $F_c \subseteq F_I$. It *partially matches* when at least half of F_c is present. A full match raises the tier; a partial match is charged to the allowance. An attacker who rewrites the message but keeps the method produces the same flags and is matched. Three of the flags (new destination, amount anomaly, memory-sourced) are computed by rule; the others are extracted by the reader and signed by the guardian.

8.3 Effect

The wallet contract reads unexpired capsules from the bulletin when it executes a request, so a capsule takes effect on the very next transaction and does not rely on the agent choosing to obey it. A wallet the threat does not apply to is unaffected and keeps its normal autonomy.

Some attacks are already stopped by the mandate: a payment to a new address needs the owner whether or not a capsule exists. There the danger is the approval itself, because this kind of fraud succeeds by persuading a person. When the owner is asked to approve, the interface shows any matching capsule, and the approval is subject to the pressure hold of Section 7.4.

8.4 Limits on capsules

A channel that can change other wallets' behaviour is itself a target, so capsules are constrained:

- *Tighten only.* A capsule can raise a tier or charge the allowance. No capsule can lower a tier, add a payee or raise a limit.
- *Bounded by confidence.* A capsule from one observer can impose at most a delay. Owner approval can be imposed only when q distinct wallets have attested the same family and features ($q = 2$ in the prototype).
- *Expiry.* Every capsule expires; the maximum lifetime is fixed in the bulletin contract. Autonomy returns without anyone acting.
- *Rate limit and identity.* Each registered wallet may publish a limited number of capsules per day, and every capsule names its issuer.
- *Owner override.* An owner may dismiss a capsule for their own wallet.

8.5 Alert first, vaccine later

The capsule acts within one transaction but is blunt and temporary. The retrained detector arrives later but recognises variants beyond the listed features and stays. A capsule's lifetime is set to outlast the next training round, so protection is continuous. Algorithm 2 shows the full response.

9. Federated Detector

9.1 Detector

Each wallet k keeps a private set $D_k = \{(x_i, y_i)\}$ of content snippets labelled benign ($y = 0$) or attack ($y = 1$). Labels come from blocked requests the owner confirmed as attacks and from requests the owner approved. A frozen text encoder ϕ maps a snippet to a vector; a small classifier with weights w outputs the score

$$s(x) = \sigma\big(w^\top \phi(x) + b\big) \in [0,1].$$

The risk band is low if $s < \tau_{\text{low}}$, high if $s \geq \tau_{\text{high}}$, and medium otherwise. Only (w, b) is trained and shared.

Algorithm 2 Immunity response to an attack on wallet a

Require: global weights w^t, wallets $1..K$, clip norm C
owner of a confirms the attack; $\alpha_a \leftarrow 0$
a adds it to D_a and publishes a capsule ▷ alert
wallets it applies to: $\alpha \leftarrow \alpha/2$; matching requests restricted
a second observer may attest, raising it to confirmed
for each wallet k in parallel **do** ▷ vaccine
    $w_k \leftarrow \textsc{LocalTrain}(w^t, D_k, E)$
    $\delta_k \leftarrow \textsc{Clip}(w_k - w^t, C)$
    send $\textsc{Mask}(n_k\,\delta_k)$ to the aggregator
end for
$w^{t+1} \leftarrow w^t + \frac{1}{n}\sum_k n_k\,\delta_k$ ▷ masks cancel
aggregator posts $H(w^{t+1})$ and $t{+}1$ to the bulletin
for each wallet k **do**
    if model hash matches bulletin **and** canary passes **then**
        adopt w^{t+1}
    else
        keep w^t and raise an alert
    end if
end for
the capsule expires; allowances refill; the detector remains

9.2 Training round

Each wallet minimises the local loss

$$F_k(w) = \frac{1}{n_k} \sum_{(x,y)\in D_k} \ell\big(s(x), y\big), \qquad n_k = |D_k|,$$

for E epochs from the current global weights, and the aggregator computes the weighted average [16]

$$w^{t+1} = \sum_{k=1}^{K} \frac{n_k}{n}\, w_k^{t+1}, \qquad n = \sum_{k=1}^{K} n_k.$$

Updates are clipped to norm C and masked pairwise so that the aggregator sees only the sum [18]. No snippet, conversation or transaction leaves a wallet.

9.3 Anchoring and adoption safeguards

The bulletin stores the hash and version of each shared model. A wallet can therefore verify that it received the model the federation agreed on, and an auditor can reconstruct which detector and which capsules governed any past transaction. Before adopting a model, each wallet evaluates it on a private *canary set*; a model that lowers recall or raises the false-positive rate beyond a tolerance is rejected. Together with clipping, this limits what a poisoning participant can do [21].

10. Implementation

Table 10 summarises the prototype. The federated layer is written against Flower for speed of development; it uses only horizontal federated averaging with secure aggregation, which FATE also provides, so it can be moved to FATE without redesign. Contracts are plain Solidity with no chain-specific features, which keeps a later port to a permissioned EVM-compatible chain such as FISCO BCOS [23] straightforward.

11. Evaluation Protocol

All measurements are produced by scripts in the repository so that reviewers can repeat them. Table 11 lists the metrics and the targets the design aims for. Targets are design goals, not results. For every task, tier path, mandate entry, capsule and model anchor, the submission lists the transaction hash and contract address.

Table 10. Prototype implementation.

Component	Implementation
Chain	Public EVM test network; test stablecoin with no monetary value
Policy wallet	Solidity smart account: signed mandate, session key, allowlist, limits, obligation registry, caution allowance, time-lock queue for payments and for owner changes, pressure hold, co-signature, freeze
Savings vault	Minimal deposit and withdraw contract, owner-only withdrawal
Threat bulletin	Registry of capsules, attestations and model hashes; enforces capsule limits
Planner, reader	Two separately prompted language-model roles; reader has no tools
Validator	Deterministic code; schema, provenance and state checks
Guardian	Encoder plus linear classifier; separate signing key
Federation	Three clients, federated averaging, pairwise masking
VaxLab	Scripted attackers plus one rewording agent; never holds a wallet key
Interface	Web chat with mandate view, financial state panel, allowance meter, alerts and considered confirmation at approval, explorer links

11.1 Headline measure

A wallet that asks the owner about everything is safe and useless; one that asks about nothing is useful and unsafe. We therefore report the *safe autonomy rate*: the share of legitimate tasks completed without the owner, counted only if no hard policy was violated anywhere in the run.

$$SAR = \frac{\text{legitimate tasks completed without the owner}}{\text{all legitimate tasks}},$$
$$\text{reported only if hard-policy violations} = 0.$$

11.2 Task and affordability suites

The task suite contains 40 instructions (ten per task, varied in wording, amounts and payees) and 10 that are ambiguous or impossible. A task succeeds if the expected on-chain state change is confirmed by the receipt; an ambiguous instruction succeeds if the agent asks or refuses with the correct reason and sends nothing. The affordability suite pairs 20 financial states with requests, covering comfortable, tight and breach cases, payment of a registered obligation, and income that has not arrived. Outcomes are compared with hand-computed answers; the rule is deterministic, so any disagreement is a defect.

11.3 VaxLab attack suite

VaxLab is a separate test harness that plays the attacker and never holds a wallet key. Scripted attackers cover five families: direct instructions, indirect instructions in retrieved content, false memory entries, encoded instructions (for example Morse code and base64), and recipient substitution. Three further scenarios test the new mechanisms: a *probing sequence* of slightly unusual small payments, a hostile wallet that publishes false capsules and poisoned updates, and a *panicked owner*, a script that approves every prompt at once. A rewording agent paraphrases every attack, so each defence is tested on versions it has not seen. An attack *succeeds* only if an unauthorised transaction is executed on chain.

Table 11. Evaluation metrics and design targets.

Metric	Definition	Target
Safe autonomy rate	Legitimate tasks done without the owner, zero violations	$\geq 80\%$
Hard-policy violations	Mandate breaches of any kind	0
Task success rate	Correct state change on chain, 40 instructions	$\geq 90\%$
Ambiguous requests handled	Clarified or refused, 10 instructions	$\geq 90\%$
Affordability decisions	Agreement with hand-computed tier, 20 scenarios	100%
Unauthorised value moved	Across all VaxLab attacks	0 beyond L1 limits
Probing sequence	Odd payments executed before the owner is asked	Within the allowance
Panicked-owner loss	Value leaving before a hold or delay ends when every prompt is approved at once	0
Capsule response	Transactions before restriction is active	0
Reworded variants matched	After one capsule	$\geq 90\%$
False-capsule harm	Rules loosened by hostile capsules	0
Cross-family recall gain	Recall on unseen families, (c) minus (b)	Positive for every wallet
Escalation and false-positive rates	Benign requests sent to the owner; escalated by the detector	Reported; $\leq 5\%$
Friction and cost	Wait imposed on legitimate new payees and urgent requests; time to full allowance; seconds and gas per task	Reported

11.4 Shared-response experiments

Capsules. Wallet A is attacked and publishes a capsule. We measure how many transactions pass before matching requests on other wallets are restricted, the share of reworded variants matched, whether a wallet outside the capsule's scope stays unrestricted, and how long autonomy takes to return. *Federation.* Each wallet is seeded with a different attack family; the others are held out. We compare (a) contract checks only, (b) plus a locally trained detector, (c) plus the federated detector, measuring each wallet's recall on families it has never seen locally.

11.5 Demonstration scenario

The live demonstration follows one story across three wallets:

1. The owner of wallet A states a mission; the mandate is shown, signed and stored on chain.
2. The agent pays rent and a hosting invoice on its own (L1).
3. Expected income is late. A 300 hosting upgrade is requested; the state engine finds it tight, and the agent holds it and explains the 120 shortfall.
4. An invoice arrives that appears to come from the hosting provider, with a new payment address, urgent wording and a request to ignore earlier details. The new address needs the owner; the owner rejects it and confirms fraud.
5. Wallet A publishes a capsule; its allowance drops to zero.
6. Wallet B receives a reworded version. The flags match and the alert is shown. B's owner, in a hurry, approves anyway. Nothing moves: the new address is subject to the loosening delay, and the owner later cancels it. Wallet C does not accept that channel and is unaffected.
7. Wallet B then receives an inflated invoice from an approved payee to its usual address. Alone it would pass; with the partial match and the halved allowance it waits.
8. A federated round runs, the new detector's hash is anchored, the capsule expires and the allowances refill.

12. Compliance

The prototype is designed to be acceptable to a regulated institution. All assets are test tokens on a test network; no token is issued and nothing is traded for profit. Each wallet has an identified owner who signs the mandate, holds the override key and is responsible for the agent's actions. Every intent, decision, guardian signature, allowance charge, hold, capsule and model version is logged, and the on-chain part is public. Conversations and transaction details stay on the wallet's own node; only fixed-field capsules, masked model updates and hashes leave it. Third-party code and models are used under their open-source licences.

13. Known Risks and Limitations

Table 12 lists the risks we are aware of. It is the basis of the security self-assessment report required by the rules.

14. Future Work

Left out of the submitted prototype: a full adversarial swarm in VaxLab with an evolutionary generator of attacks; reputation for capsule issuers and resistance to many fake wallets; a mutual-aid pool that compensates verified losses [22]; deployment on a permissioned chain; income forecasting and differential privacy for updates.

15. Conclusion

AgentVax is a self-immunising wallet agent that adjusts how much financial autonomy it exercises while its authority stays under limits only the owner can change. Usefulness comes from a mission the owner states once and tasks that complete on chain. Bounds come from a mandate enforced by the contract and a guardian signature the model cannot forge. Prudence comes from asking whether the owner can afford a payment and how much uncertainty has already been accepted today. Learning comes in two speeds: a capsule that makes exposed wallets cautious at once, and a jointly trained detector that keeps them protected. The owner is protected as well: authority can be reduced in a moment and extended only with time. The evaluation protocol makes each of these claims measurable.

References

[1] Organizing Committee, "International Track: Topics & Rules," 2026 Shenzhen International FinTech Competition (FinTechathon), 2026. `http://fintechathon.g-ican.com`

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

Table 12. Known-risk list.

ID	Risk	Consequence	Mitigation and residual exposure
R1	Undetected injection	Planner proposes an attacker-chosen action	Contract bounds the outcome: L1 payments to approved payees within the limits, the discretionary amount and the day's allowance
R2	Guardian key compromise	Attacker can authorise L1 execution and understate flags	Key held in a separate process; limits, affordability and rule-computed flags still apply; owner can rotate the key and freeze the wallet
R3	Reader misses a flag	A capsule fails to match, or the allowance is undercharged	Three flags are computed by rule and cannot be suppressed; a missed flag returns the request to the ordinary tier rule, not to no rule
R4	Owner approves a fraud under pressure	Funds leave with valid authorisation	New payees and higher limits wait for the loosening delay; pressured approvals are held; cancelling is instant. Residual: an owner still deceived when the delay ends
R5	Delays obstruct a genuine emergency	A legitimate urgent payment is late	The owner sets the delays while calm; a trusted co-signer can release them; the cost is measured and reported
R6	Incomplete obligation registry	A bill nobody registered is not protected	Schedules register automatically; the agent proposes obligations it finds in invoices; the reserve provides a margin
R7	Arbitrary allowance values	Too strict wastes the owner's time; too loose admits probing	Owner-set integers with stated defaults; escalation rate and probing result reported
R8	False or excessive capsules	Honest wallets are slowed (denial of service)	Tighten-only; single-observer capsules limited to a delay; rate limit; expiry; owner may dismiss
R9	Small federation	With three clients, two colluding parties can infer the third party's update	Stated limitation of the prototype; privacy improves with more participants
R10	Model poisoning by a participant	Shared detector degrades or learns a blind spot	Update clipping; canary test before adoption; on-chain version history allows rollback
R11	False positives	Legitimate payments are delayed or need approval	Thresholds tuned on benign data; L2 delays, not blocks; rates reported
R12	Compromised approved payee	Funds sent to a legitimate but hijacked payee	Limits, usual-amount flag, discretionary amount and allowance limit exposure
R13	Hosted language model; testnet conditions	Outage breaks new requests; fees and finality are not representative	Scheduled payments run without the model; contracts avoid chain-specific features

[3] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025.

[4] OECD.AI Policy Observatory, "AI prompt injection exploit drains Grok-linked crypto wallet," AI Incidents and Hazards Monitor, May 2026. https://oecd.ai/en/incidents/2026-05-04-4a73

[5] Giskard, "How Grok got prompt-injected: An X user drained $150,000 from an AI wallet," giskard.ai, 2026.

[6] A. S. Patlan, P. Sheng, S. A. Hebbar, P. Mittal and P. Viswanath, "Real AI agents with fake memories: Fatal context manipulation attacks on Web3 agents," arXiv:2503.16248, 2025.

[7] T. Davidovic and H. Tourpe, "How agentic AI will reshape payments," IMF Note 2026/004, International Monetary Fund, 2026.

[8] The Block, "Human player outwits Freysa AI agent in $47,000 crypto challenge," Nov. 2024.

[9] AI Incident Database, "Incident 1003" (AIXBT autonomous agent), 2025. https://incidentdatabase.ai/cite/1003

[10] The Block, "AI agent created by OpenAI dev 'accidentally' sends entire memecoin holdings to reply guy," Feb. 2026.

[11] V. Buterin *et al.*, "ERC-4337: Account abstraction using alt mempool," Ethereum Improvement Proposals, 2021.

[12] "ERC-8004: Trustless agents," Ethereum Improvement Proposals, 2025.

[13] S. Willison, "The dual LLM pattern for building AI assistants that can resist prompt injection," simonwillison.net, Apr. 2023.

[14] E. Debenedetti *et al.*, "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[15] OASIS, "STIX Version 2.1," OASIS Standard, 2021.

[16] B. McMahan, E. Moore, D. Ramage, S. Hampson and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[17] Q. Yang, Y. Liu, T. Chen and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intelligent Systems and Technology*, vol. 10, no. 2, 2019.

[18] K. Bonawitz *et al.*, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM CCS*, 2017.

[19] Y. Liu, T. Fan, T. Chen, Q. Xu and Q. Yang, "FATE: An industrial grade platform for collaborative learning with data protection," *J. Machine Learning Research*, vol. 22, no. 226, 2021.

[20] D. J. Beutel *et al.*, "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.

[21] P. Blanchard, E. M. El Mhamdi, R. Guerraoui and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017.

[22] Hu and Chen, "Insured agents: A decentralized trust insurance mechanism for agentic economy," arXiv:2512.08737, 2025.

[23] FISCO BCOS Open Source Community, "FISCO BCOS: An enterprise-level financial blockchain platform," open-source software, github.com/FISCO-BCOS.

[24] Monetary Authority of Singapore and Association of Banks in Singapore, "MAS and ABS announce measures to bolster the security of digital banking," 19 Jan. 2022.

[25] HM Treasury, "Payment Services (Amendment) Regulations 2024," draft legislation published Mar. 2024.

[26] G. Bryan, D. Karlan and S. Nelson, "Commitment devices," *Annual Review of Economics*, vol. 2, 2010.